

AI ASSISTANT CODING

ASSIGNMENT-14.5

Name: N. Hasini

Ht.no:2303A510G3

Batch:03

Task 1 – Image Gallery Website Create
an image gallery web page.

Requirements:

- Display images in a grid layout.
- Use AI to:
- Optimize layout using CSS Grid.
- Add hover zoom or overlay effects.
- Ensure mobile responsiveness.

Prompt: Create a responsive image gallery using HTML and CSS Grid with a hover zoom effect.

Code:

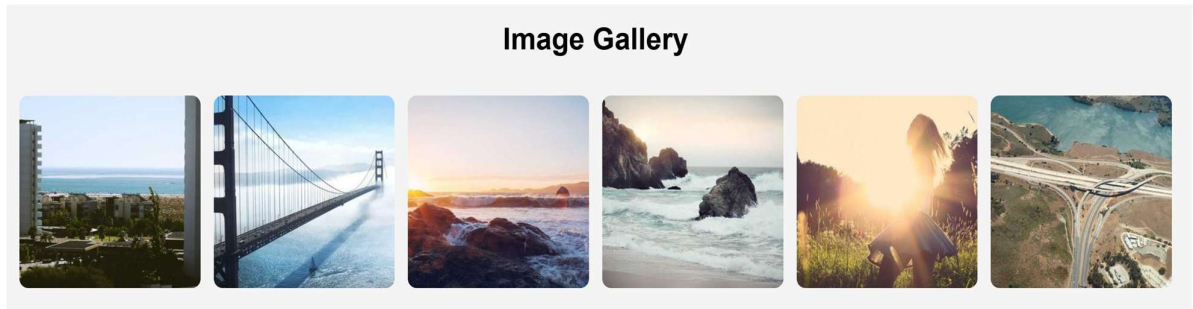
```
<!DOCTYPE html>
<html>
<head>
<title>Image Gallery</title>
<style>
body{
font-family: Arial;
text-align:center;
background: #f4f4f4;
}
.gallery{
display:grid;
grid-template-columns:repeat(auto-fit,minmax(200px,1fr));
gap:15px;
padding:20px;
}
.gallery img{
width:100%;
border-radius:10px;
transition:0.3s;
}
.gallery img:hover{
transform:scale(1.1);
}
</style>
</head>
<body>
<h1>Image Gallery</h1>
<div class="gallery">






</div>
</body>
</html>
```

Output:



Explanation:

The image gallery displays images using CSS Grid layout.

The grid automatically adjusts columns depending on screen size, making it responsive.

A hover zoom effect is applied using CSS transform to enlarge images when the mouse pointer moves over them.

Task 2 – Profile Card Generator

Create a profile card web page.

Requirements:

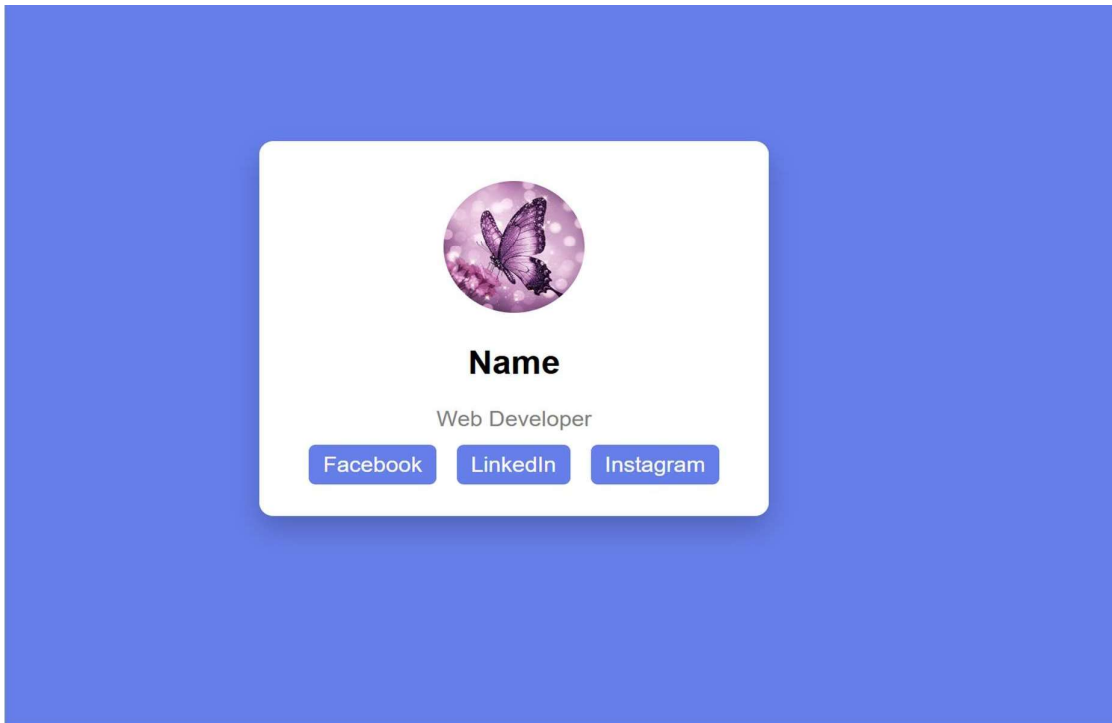
- Display user image, name, role, and social media links.
- Use AI to:
- Suggest modern UI colors and fonts.
- Design the card using Flexbox.
- Add hover animation and shadow effects

Prompt: Create a modern profile card using HTML and CSS Flexbox with hover animation.

Code:

```
<!DOCTYPE html>
<html>
<head>
<title>Profile Card</title>
<style>
body{
font-family: Arial;
background: #667eea;
display: flex;
justify-content: center;
align-items: center;
height: 100vh;
}
.card{
background: white;
padding: 30px;
text-align: center;
border-radius: 10px;
box-shadow: 0 10px 20px rgba(0,0,0,0.2);
transition: 0.3s;
}
.card:hover{
transform: translateY(-10px);
}
.profile{
width: 100px;
border-radius: 50%;
}
.role{
color: gray;
}
.social a{
text-decoration: none;
margin: 5px;
padding: 6px 10px;
background: #667eea;
color: white;
border-radius: 5px;
}
</style>
</head>
```

Output:



Explanation:

The profile card shows a user image, name, role, and social media links.

The layout is styled with modern colors and uses Flexbox to center the card. A hover animation moves the card slightly upward and a shadow effect gives it a modern UI appearance.

Task 3 -Develop a dynamic shopping cart system.

Requirements:

- Add, remove, and update product quantities.
- Display real-time total price.
- Use AI to:
 - Manage cart state using JavaScript objects.
- Persist cart data in localStorage.

Prompt: Build a dynamic shopping cart using HTML, CSS, and JavaScript with localStorage.

Code:

```

<!DOCTYPE html>
<html>
<head>
<title>Shopping Cart</title>
<style>
body{
font-family:Arial;
text-align:center;
background: #f4f4f4;
}
.product{
background: white;
padding:15px;
margin:10px;
display:inline-block;
border-radius:10px;
box-shadow:0 5px 10px rgba(0,0,0,0.2);
}
button{
padding:5px 10px;
margin:5px;
background: #4CAF50;
color: white;
border:none;
border-radius:5px;
cursor:pointer;
}
</style>
</head>
<body>
<h1>Shopping Cart</h1>
<div class="product">
<h3>Laptop</h3>
<p>$800</p>
<button onclick="addItem('Laptop',800)">Add</button>
</div>
<div class="product">
<h3>Phone</h3>
<p>$500</p>
<button onclick="addItem('Phone',500)">Add</button>
</div>

```

Output:



Explanation:

The shopping cart allows users to add products and view them in the cart.

JavaScript objects store product details such as name, price, and quantity.

The total price updates dynamically whenever products are added or removed.

Cart data is stored in localStorage, ensuring that items remain saved even after refreshing the page.

Task 4 – Role-Based Access Control Application

Create a role-based web application.

Requirements:

- Admin, Editor, User roles.
- Conditional UI rendering.
- Use AI to:
- Design access control logic.
- Maintain secure state transitions.
- Create scalable UI components.

Prompt: Create a role-based web app with Admin, Editor, and User roles using JavaScript.

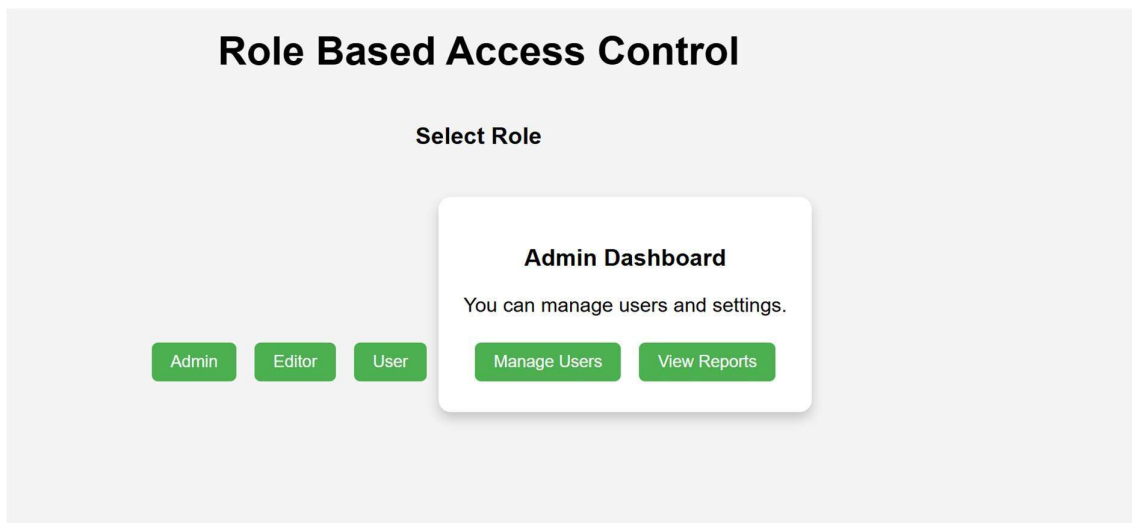
Code:

```

<!DOCTYPE html>
<html>
<head>
<title>Role Based Access Control</title>
<style>
body{
font-family: Arial;
text-align:center;
background: #f4f4f4;
}
.container{
margin-top:40px;
}
button{
padding:8px 15px;
margin:5px;
background: #4CAF50;
color: white;
border:none;
border-radius:5px;
cursor:pointer;
}
.panel{
margin-top:20px;
padding:20px;
background: white;
display:inline-block;
border-radius:10px;
box-shadow:0 5px 10px rgba(0,0,0,0.2);
}
</style>
</head>
<body>
<h1>Role Based Access Control</h1>
<div class="container">
<h3>Select Role</h3>
<button onclick="setRole('Admin')">Admin</button>
<button onclick="setRole('Editor')">Editor</button>
<button onclick="setRole('User')">User</button>
<div class="panel" id="panel">
<p>Please select a role</p>

```

Output:



Explanation:

This application demonstrates role-based access control with Admin, Editor, and User roles.

Each role sees different interface options using conditional UI rendering. JavaScript controls the access logic to ensure users only see features allowed for their role.

Task 5 – Multi-Language Website

Design a multilingual website.

Requirements:

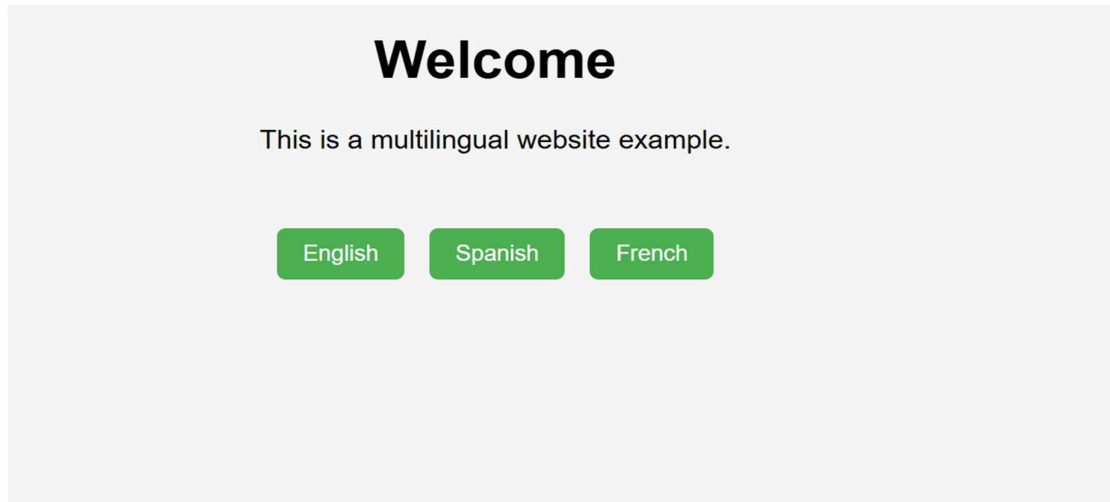
- Switch languages dynamically.
- Persist language preference.
- Use AI to:
 - Structure translation files.
 - Handle RTL/LTR layouts.
 - Improve accessibility

Prompt: Create a multilingual website with dynamic language switching using JavaScript.

Code:

```
<!DOCTYPE html>
<html>
<head>
<title>Multi Language Website</title>
<style>
body{
font-family: Arial;
text-align:center;
background: #f4f4f4;
}
.container{
margin-top:40px;
}
button{
padding:8px 15px;
margin:5px;
background: #4CAF50;
color: white;
border:none;
border-radius:5px;
cursor:pointer;
}
</style>
</head>
<body>
<h1 id="title">Welcome</h1>
<p id="text">This is a multilingual website example.</p>
<div class="container">
<button onclick="setLanguage('en')">English</button>
<button onclick="setLanguage('es')">Spanish</button>
<button onclick="setLanguage('fr')">French</button>
</div>
<script>
const translations = {
en:{
title:"Welcome",
text:"This is a multilingual website example."
},
es:{
title:"Bienvenido",
text:"Este es un ejemplo de sitio web multilingüe."
}
```


Output:



Explanation:

This multilingual website allows users to switch between different languages dynamically.

Translations are stored in JavaScript objects, making it easy to manage multiple languages.

The selected language is saved using localStorage, so the preference remains after refreshing the page.

This approach improves accessibility and usability for users from different language backgrounds.

Task 6 – Online Examination System Create an online exam interface.

Requirements:

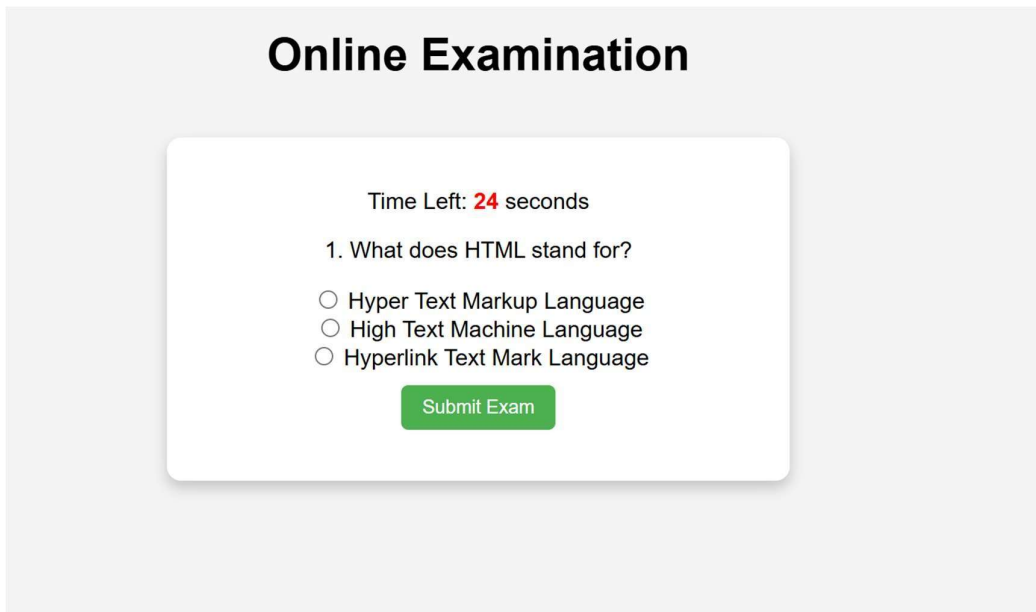
- Timer, questions, and submission.
- Prevent multiple submissions.
- Use AI to:
- Manage exam state.
- Improve exam UX.
- Ensure accessibility.

Prompt: Create an online exam interface with questions, timer, and submission using HTML, CSS, and JavaScript.

Code:

```
<!DOCTYPE html>
<html>
<head>
<title>Online Exam</title>
<style>
body{
font-family: Arial;
text-align:center;
background: #f4f4f4;
}
.exam-box{
background: white;
padding:20px;
margin:40px auto;
width:400px;
border-radius:10px;
box-shadow:0 5px 10px rgba(0,0,0,0.2);
}
button{
padding:8px 15px;
margin-top:10px;
background: #4CAF50;
color: white;
border:none;
border-radius:5px;
cursor:pointer;
}
#timer{
font-weight:bold;
color: red;
}
</style>
</head>
<body>
<h1>Online Examination</h1>
<div class="exam-box">
<p>Time Left: <span id="timer">30</span> seconds</p>
<p>1. What does HTML stand for?</p>
<label><input type="radio" name="q1"> Hyper Text Markup Language</label><br>
<label><input type="radio" name="q1"> High Text Machine Language</label><br>
<label><input type="radio" name="q1"> Hyperlink Text Mark Language</label><br>
```

Output:



Explanation:

This online examination system displays questions with a countdown timer. Users can answer the question and submit the exam. The timer automatically submits the

exam when time ends. JavaScript manages the exam state and prevents multiple submissions, improving the user experience and accessibility.